2026 IEEE 23rd International Conference on Software Architecture Companion (ICSA-C) | 979-8-3315-7787-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICSA-C68850.2026.00041

A No-Code Composable Pipeline for Holistic Microservice Assurance

Shakthi Weerasinghe
Department of Electrical and Computer Engineering
University of Arizona
Tucson, USA
syweerasinghe@ariozn.edu

***Abstract*—Modern cloud-native "systems" are distributed abstractions scattered across independent repositories. While this multi-repository, loosely-coupled approach maximizes agility, it exposes a fundamental quality assurance gap: existing CI/CD tools are confined to single-repository contexts and linear procedural execution, leaving architects with no mechanism to reason about system-wide health. Thus, cross-service architectural drift and latent security violations go undetected by design. This paper introduces the Holistic Analysis Orchestrator (HAO), a novel framework that shifts the paradigm from linear pipelines to assurance graphs. HAO's "no-code" visual wizard lets practitioners compose multi-repository verification strategies from disparate analysis engines–without scripting. We demonstrate a God's Eye workflow that ingests an entire distributed topology to concurrently perform Change Impact Analysis (CIMET), Formal Authorization Policy Consistency Verification (via SMT solvers), Neuro-Symbolic Latent Risk Assessment (Aegis), and Call-graph-Aware Test Suite Generation (AITest). The result is a feedback-driven assurance fabric that unifies static analysis, formal logic, and generative validation—capabilities absent from any existing enterprise tooling.**

***Index Terms*—Microservice Orchestration, Multi-Repository Analysis, Graph-Based Assurance, Formal Verification, Change Impact Analysis, Generative AI Testing, No-Code CI/CD**

I. Introduction

The widespread adoption of cloud-native technologies [1], [2] has driven a paradigm shift toward decentralized microservice (MS) architectures, where functionality relies on complex interactions among independent, polyglot services [3]. While this decoupling facilitates immense agility, it inadvertently decouples assurance, creating profound "visibility gaps" in MS system quality [4]–[6]. In modern multi-repository environments, MS architecture never tangibly exists in a persistent form; it is a transient, emergent property of hundreds (e.g., 700+ in Netflix), or even thousands (e.g., ∼2,200 in Uber) of independent services and CI/CD pipelines [7]. Consequently, identifying and prioritizing risks requires tracing logic across complex network boundaries [8], "every time".

This architectural fragmentation creates critical blind spots. Take TrainTicket[1] system for instance. A developer modifying a data schema or an API contract in a downstream provider (e.g., `ts-train-service`) lacks the global visibility to determine if the change implicitly: (i) violates an (up-stream) authorization policy in the `ts-preserve-service` [9], or (ii) introduces latent security vulnerabilities that only manifests when combined with specific business logic elsewhere in the mesh. Existing industrial practices rely on siloed point solutions: Static Application Security Testing (e.g., Snyk[2] and SonarQube[3]) for local code flaws, Search-Based Software Testing (e.g., EvoMaster [10], RestTestGen [11], and RESTler [12]) for automated validation, Application Performance Monitoring (e.g., Datadog[4], New Relic, and Dynatrace) for runtime errors. Yet, manual code reviews are necessary to inspect distributed logic because these tools are effectively shallow. State-of-the-art SBST tools typically operate at the endpoint level (e.g., fuzzing `/orderservice/order`), oblivious to the multi-hop call chains a request triggers across the mesh. Without systematic tooling capable of reasoning about transitive dependencies [13], these disparate tools fail to capture the holistic, interdependent reality of the distributed architecture.

Therefore, the primary **research question** is:
How can disparate analysis, verification, and validation techniques be orchestrated to perform holistic, architectural-level assurance for complex, evolving, multi-repository MS systems?

To bridge this gap, I present the novel **Holistic Analysis Orchestrator** (HAO), a composable analysis environment that orchestrates disparate verification and validation primitives to construct a God's Eye View of complex MS meshes.

Unlike rigid scanning tools, HAO ingests multi-repository codebases to build a normalized Intermediate Representation (IR)–a unified graph substrate capturing topology, logic, and data flows. This substrate enables architects and engineers to mix and match diverse analysis capabilities including change impact analysis [14], formal verification of distributed policy [15], neuro-symbolic introspection for latent vulnerabilities [13], and validative, distributed call graph-ware test generation into composable, **no-code workflows**. By providing a single pane of glass for system-wide analysis, HAO enables a vital shift from shallow, context-bounded checks to rigorous, deep architectural assurance.

II. A Composable Assurance Environment

We propose a paradigm shift from rigid, monolithic scanning tools to a composable assurance environment orchestrated

[1]https://github.com/FudanSELab/train-ticket

[2]https://snyk.io/product/snyk-code/
[3]https://www.sonarsource.com/products/sonarqube/
[4]https://www.datadoghq.com

2768-4288/26/$31.00 ©2026 IEEE
DOI 10.1109/ICSA-C68850.2026.00041

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.

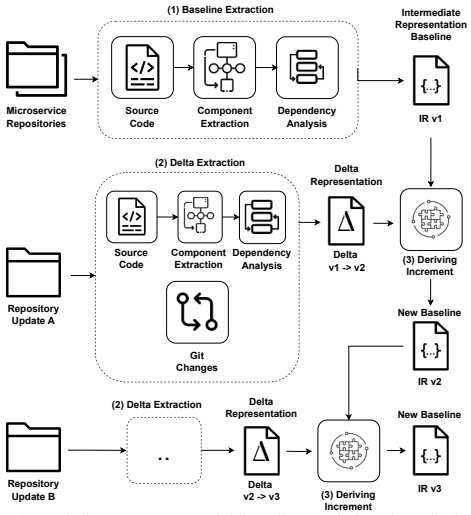


Fig. 1. ISAR derives system model baseline and then iteratively improves it with changes in microservice codebases. The change deltas encapsulate changes to determine the change span across the program slices.

through HOA–a specialized **MS Analysis Pipeline Creator**. In this model, assurance is not a fixed, linear process but a programmable workflow constructed from interoperable analysis primitives. HOA provides a visual orchestration environment[5] where complex analytical tasks are abstracted into modular components. By linking these modules, architects can construct "self-driving" quality gates that are tailored to the specific topological and risk profiles of either the *entire* (configuring for all MS system repositories) OR focal *sub meshes* (selected repositories). The HOA architecture wizard is organized into a cohesive data substrate, an intelligence layer of specialized engines, and a control layer for orchestration.

A. Foundation Module: Incremental Architecture Reconstruction (ISAR)

A primary module in any analysis configuration is the ISAR primitive [9], [14] which serves as the data substrate to resolve multi-repository fragmentation, where the global topology is often emergent rather than explicit. Unlike traditional scanners that require exhaustive re-indexing, ISAR achieves scalability through a "baseline-and-delta" mechanism (Figure 1). It extracts local Component Call Graphs from individual repositories and stitches them into a global System Dependence Graph (SDG) by resolving remote calls and data overlaps. The output is a normalized unified Intermediate Representation (IR)–a JSON-based hypergraph exposing architectural invariants. Recognizing 96-100% of components in TrainTicket at a recall 86-100% [16], this IR serves as the universal data language for all subsequent pipeline modules.

B. The Evolutionary Module: Change Impact Analysis (CIA)

To reason about system evolution, the pipeline incorporates a CIA primitive. Linked with the ISAR module, CIA performs

[5]Demonstration: https://bit.ly/cloudhub-playlist

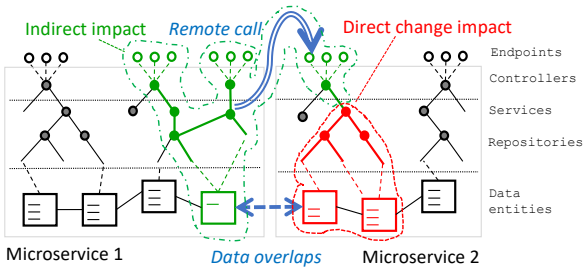


Fig. 2. Illustration of the blast radius–the impact of a change in a service.

differential analysis on the SDG to calculate the Blast Radius of a code modification (Figure 2). It distinguishes between direct and transitive (indirect) impact, identifying how a local change propagates across distributed service boundaries. By applying topological conflict detection rules, the module flags violations such as invalid calls and service method modifications. This intelligence is rendered as a dynamic, force-directed 3D graph (Figure 3), facilitating developers to perceive implicit cross-service dependencies normally invisible during manual reviews.

C. The Logical Correctness Module: Formal Verification

The pipeline achieves mathematical rigor through the Formal Verification module [15], which translates the Unified IR into First-Order Logic constraints for Satisfiability Modulo Theories (SMT) solvers. The engine enforces three critical invariants: (i) *endpoint security* (privilege requirements), (ii) *propagation consistency* (role subset constraints along call edges), and (iii) *data access uniformity* (preventing "backdoors" via relaxed permissions on alternate ingress paths). This SMT-based approach scales linearly with system size, typically under 3.2 seconds[6]. Upon detecting an inconsistency, the solver outputs an optimal set of suggestions to restore holistic policy consistency (Figure 4).

D. Validation Strategy: Targeted Multi-Level Test Generation

While mathematically rigorous, proofs must be validated by executable reproduction scenarios. Standard test generators (e.g., EvoMaster, RestTestGen) are typically oblivious to the broader architecture, fuzzing only the endpoints (entry points) in isolation without context. This primitive, named AITest[7], overcomes this limitation by translating the IR's architectural clues and distributed security policy (from code) into a multi-dimensional matrix. The *Scenario Generator* leverages this matrix to synthesize dependency-aware risk scenarios, which are translated into executable test classes (in Java, Python, or Bash scripts containing cURL) using a Large Language Model of choosing.

Crucially, the engine bridges natural language intent and functional code via adaptive prompting. It dynamically interleaves *Role-Play* to simulate user personas, *Few-Shot* examples

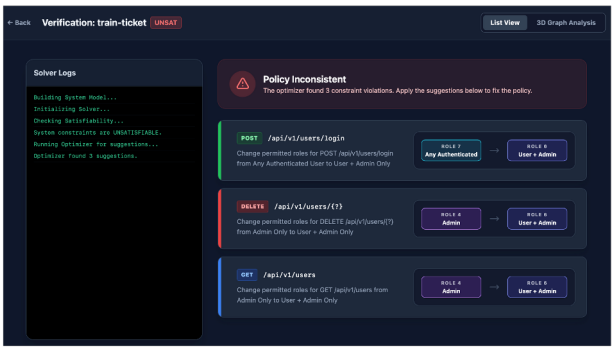
[6]Artifacts: https://github.com/cloudhubs/authorization-consistency-solver
[7]Repository: https://github.com/cloudhubs/aitest-scripts.git

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.

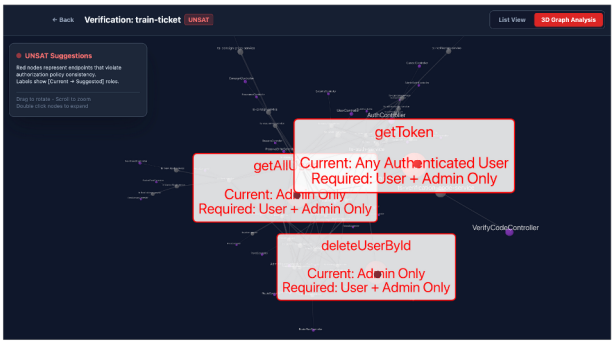


(a) (b)

Fig. 3. Microservice SDG illustrated as force-directed 3D graph at subsequent commits: (a) `a4ed2433b0b6ab6e0d60115fc19efecb2548c6cd` with anti-pattern highlights, and (b) `313886e99befb94be6cd45f085c98e0019f59829` with change impacts–retractions (in red) & inclusions (in green).



(a)



(b)

Fig. 4. The authorization policy consistency suggestions from formal verification for TrainTicket in: (a) list view and (b) graph view to identify impacted services.

for syntactic compliance, and *Chain-of-Thought* reasoning to decompose complex transitive states into valid API sequences. AITest produces 90-97.4% runnable, semantically valid authorization drift detecting tests[8], while generating payloads that satisfy endpoint constraints and mocking deep-chain database states; thus, moving beyond shallow "200 OK" checks that validate distributed business logic.

E. Risk Quantification: Multi-Source Evidence Fusion

In our current version, the Neuro-Symbolic Latent Risk Calculus (Aegis) [13], transforms distributed security diagnostics by replacing manual "guess work" [5] with mathematical rigor. Instead of overwhelming engineers with scalar alerts, this module employs Subjective Logic [17] to mathematically fuse four orthogonal signal streams–structural, semantic, business context, and dynamic evidence, into multi-dimensional *Opinions* $\omega = (b, d, u, a)$ that explicitly quantify uncertainty (u). In empirical evaluations [13], Aegis successfully identified latent vulnerabilities in execution paths that were deemed *clean* by industry-standard tools, i.e., SonarQube, demonstrating the necessity of context-aware fusion[9].

This facilitates modeling the security posture of the entire MS system in an simplified, intuitive interface (Figure 5). By applying specialized operators such as *Cumulative Fusion* ($\oplus$) to resolve conflicting signals and *Trust Discounting* ($\otimes$) to penalize uncertainty in evidence extraction models, the calculus filters out architectural noise. This significantly reduces architect cognitive load necessary for deep, prognostic analysis and enables *Abductive Reasoning* ($\bar{\odot}$) to autonomously trace runtime symptoms to structural root causes (Figure 6).

F. The Orchestration Core: Holistic Analysis Orchestrator

These disparate mathematical engines in the current version, while powerful individually, require a unifying control plane. We resolve this by encapsulating these primitives within the **Holistic Analysis Orchestrator** (HAO), the visual centerpiece of our framework (Figure 7). HAO transforms abstract mathematical engines described above into modular, interoperable *analysis cards* placed upon a programmable digital canvas. The implementation can be found in GitHub[10].

[8]Artifacts: https://doi.org/10.5281/zenodo.19199479

[9]Artifacts: https://doi.org/10.5281/zenodo.18307907

[10]https://github.com/UACloudVision/mvp-pipeline-client.git

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.

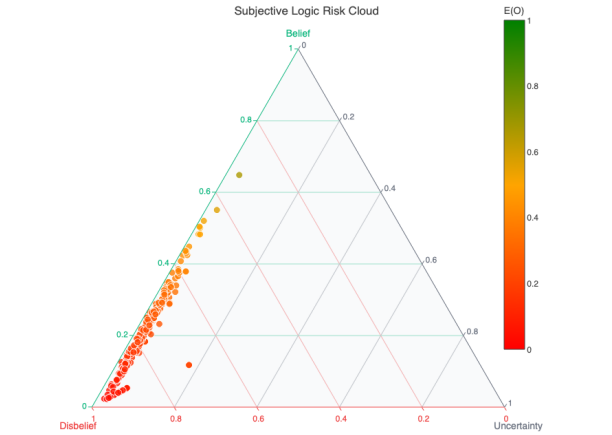


Fig. 5. The 2D latent risk cloud of TrainTicket system. Each point is an risk opinion of a distributed path derived from fusing multiple evidence.

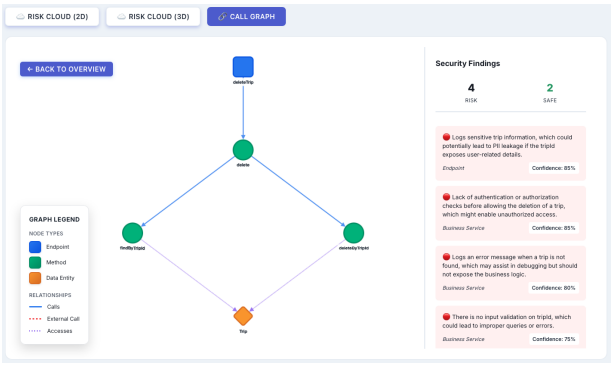


Fig. 6. Clicking on a point in the 2D/3D risk cloud, directs to the distributed call graph visualization with potential latent vulnerabilities, prioritized by confidence.

In this environment, the architect acts as a composer rather than a script-writer. Users can freely interlink, or criss-cross, analysis cards to construct complex, multi-dimensional workflows. For instance, as depicted in Figure 7, a architect can link the *ISAR card* (data substrate) to *Formal Verification*, and then route those verification results directly into the *Scenario Generator* card to automatically produce targeted validations for identified policy violations. Further, by simply linking *Formal Verification* card to the *Aegis* card, users can pass a new stream of evidence to construct a more context-aware latent risk modeler. Thus, by constructing complex directed acyclic graph of assurance, architects prompt the HAO to transparently handle all complex data marshaling including translating the JSON-based unified IR into SMT constraints, Subjective Logic opinions, or test matrices, effectively abstracting the mathematical complexity of the underlying intelligence modules.

This "glass-box" approach democratizes high-assurance engineering, allowing even non-experts to construct self-driving quality gates that trigger deep architectural scans upon every commit. Whether configuring an analysis pipeline/graph for the *entire* system topology or a *sub-mesh* of critical repositories, the HAO provides the comprehensive visibility necessary to govern complex, distributed architectures.

III. Motivating Case Study: The Anatomy of a Silent Drift

To demonstrate the necessity of a holistic, graph-native assurance ecosystem, I examined a representative *silent drift* scenario using the TrainTicket benchmark–a widely adopted MS system consisting of 41 distinct services. Hence, this case study illustrates how the interoperability of our ISAR, system visualization, formal verification, test generation, and risk calculus modules in the current version 1.1.0 detects complex architectural regression that standard siloed tools miss.

A. The Distributed Topology

Let us focus on a critical sub-graph involving three services: `ts-preserve-service`, which handles ticket booking for regular users (`USER` role); `ts-admin-route-service`, which manages train route logistics for the `ADMIN` role; and `ts-train-service`, a shared downstream provider supplying train metadata.

The architectural invariant mandates strict data segregation: ts-preserve-service is permitted to fetch public train schedules, while ts-admin-route-service accesses privileged operational data. Both services consume the same `GET /api/v1/trainservice/trains/id` endpoint from ts-train-service, however, the underlying Data Transfer Object is historically scoped to return only public fields (e.g., trainType, startTime).

B. A Localized Modification - But A System Evolution

Consider a developer modifying `ts-train-service` by adding `operational_cost`–a sensitive financial data field to `TrainType` entity. To facilitate rapid debugging of this new feature, let us consider that the developer temporarily relaxes the endpoint's security annotation from `@PreAuthorize("hasAnyRole('ADMIN','USER')")` to `@PermitAll`, under the assumption that ts-train-service is internal and protected by the gateway.

C. The Failure of Siloed Assurance

Under existing industrial practices, this change will pass all standard quality gates. SAST tools, e.g., SonarQube, scanning the `ts-train-service` repository will finds no syntax errors or obvious vulnerabilities; exposing a cost field is not inherently malicious in isolation. Unit tests will pass because the endpoint functions correctly. Consumer-Driven Contract Tests (e.g., using Pact[11]) generally validate that expected fields are present, not that unexpected fields are absent. Consequently, the upstream `ts-preserve-service` continues to function, unknowingly deserializing and potentially forwarding the sensitive `operational_cost` data in its own responses to the public UI. This creates a transitive data leak, where a change in a backend service implicitly compromises the confidentiality integrity of the public-facing edge service.

[11]https://docs.pact.io

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.

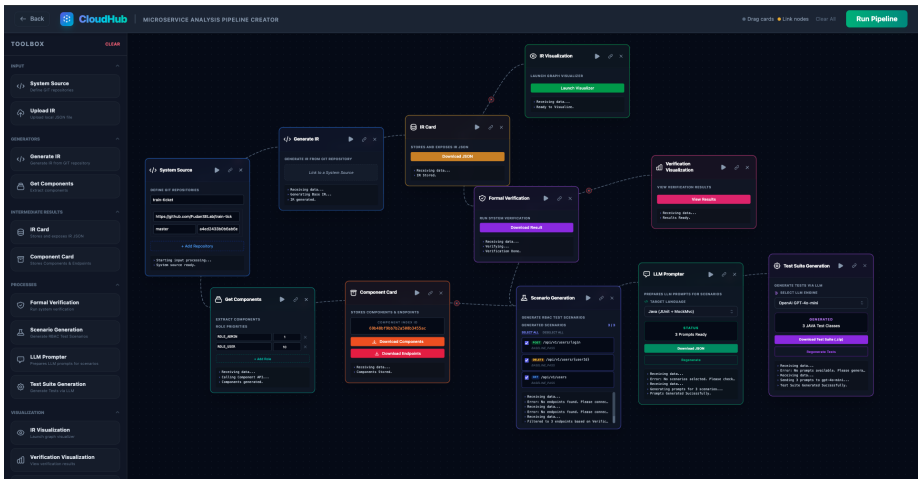


Fig. 7. The Holistic Analysis Orchestrator (HAO) canvas depicting a complex analysis graph after execution. Architects visually compose assurance pipelines by linking interoperable analysis cards, bridging the gap between code scanning and architectural reasoning.

D. Holistic Resolution via the Assurance Ecosystem

HAO can be used to detect this multi-hop failure through an orchestrated workflow as depicted in Figure 7. First, upon a commit to the `ts-train-service`, the ISAR engine will extract the delta (in the *Generate IR* card). It will identify two critical mutations: the structural expansion of the TrainType DTO and the relaxation of the security policy. The Resolution Phase updates the unified IR, maintaining the persistent edges from both ts-preserve-service and ts-admin-route-service. This will be stored in the *IR Card*.

Next, the CIA engine calculates the blast radius, visualizing the propagation path. The Force-Directed Graph highlights that ts-preserve-service (a generic user entry point) has a direct dependency on the modified artifact (when the user clicks on the "Launch Visualization" on *IR Visualization* card). This flags the potential for a ripple effect where public APIs might unintentionally expose the new internal data schema.

By linking the *IR Card* with *Formal Verification* card (as in Figure 7), the SMT solver evaluates the global authorization policy. It detects a violation of the Data Access Uniformity constraint ($Access(D) \Rightarrow R_{context} \geq P_{data}$). The solver identifies a valid execution path: User $\rightarrow$ ts-preserve-service $\rightarrow$ ts-train-service $\rightarrow$ `operational_cost`. Hence, the will solver return `UNSAT` - unsatisfied and possibly suggesting elevating the authorization policy.

This could be passed down to Aegis, to quantify and validate the latent vulnerability risk by atleast combining structural evidence (high degree centrality of ts-train-service) and business context evidence (financial nature of the added field). The result is a risk Opinion with high *belief* (b), triggering an immediate blockage of the deployment pipeline before the vulnerability can reach production.

IV. Industrial Impact

The implementation of the HAO represents a fundamental shift from disparate point solutions to a unified assurance control plane. Even in the current-state-of-the-art, industrial software evolution has been bifurcated. Independent teams utilize isolated tools for static analysis, dynamic monitoring, and architectural compliance, resulting in a fragmented view of system health. HAO resolves this gap by introducing **composable assurance**–the ability to treat security and architectural integrity not as fixed checkpoints, but as programmable workflows. By abstracting complex verification logic into interoperable nodes within the pipeline creation canvas, HAO enable teams to intuitively define, version, and execute "**Declarative Assurance Policies**" that automatically adapt to the emergent topology of their systems.

A primary industrial impact is the **democratization of architectural reasoning**. Currently, rigorous techniques such as formal verification or transitive dependency analysis are the domain of specialized experts, creating bottlenecks in rapid delivery cycles. HAO eliminates this *expertise gap* by encapsulating these advanced mathematical primitives within a **no-code orchestration environment**. This allows system architects—regardless of their background in formal methods, artificial intelligence, or any other advanced areas—to visually construct sophisticated, self-driving quality gates. These gates do not merely scan for bugs; they enforce architectural invariants directly within the CI/CD pipeline, effectively shifting architectural governance left.

Further, the platform transforms the economics of system maintenance through **context-aware precision**. In traditional environments, *alert fatigue* arises because tools lack the global context to differentiate between a theoretical flaw and a genuine business risk. By orchestrating the fusion of structural, semantic, business, and runtime evidence into a single pipeline, the HAO provides a holistic filter that technically valid but architecturally unreachable vulnerabilities. This converts the "firehose" of static alerts into a prioritized stream of actionable intelligence, allowing engineering teams to transition from reactive patching to proactive, managed system evolution

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.

without compromising developer velocity.

Looking forward, HAO's extensible canvas paves the way for a true "one-stop-shop" for holistic architectural governance. Future versions will incorporate more advanced analysis features as plug-and-play cards, allowing architects to create more complex, multi-dimensional analysis graphs. For instance, linking the IR card to a future *Data Privacy Tracker* card and a *Chaos Engineering* card may automatically validate whether fallback network routes during a service outage inadvertently leaks PIIs. Similarly, linking with a *Cloud Cost Profiler* card may predict the financial blast radius of introducing a new multi-hop transaction. By continuously expanding this ecosystem ensures that assurance pipelines can dynamically evolve to meet emerging industrial challenges.

V. Conclusion

As microservice architectures grow in complexity, the industrial reliance on fragmented, repository-scoped assurance tools has become a critical liability. This paper proposes a paradigm shift toward a composable assurance ecosystem, replacing monolithic scanners with interoperable, graph-native analysis primitives. By establishing an Incremental Software Architecture Reconstruction substrate, we demonstrated the feasibility of maintaining a near real-time, high-fidelity model of a distributed system's topology, successfully resolving the "invisible" dependencies that elude standard static analysis.

Our evaluation confirms that orchestrating these primitives—coupling the logical rigor of SMT-based formal verification with the contextual intelligence of the Neuro-Symbolic Risk Calculus and the validation capacity of generative test suites—creates an assurance posture far greater than the sum of its parts. This synergy enables the detection of silent drifts and transitive authorization violations that remain invisible to siloed point solutions. Furthermore, encapsulating these advanced capabilities within a **no-code orchestration ecosystem** provides a pragmatic path toward democratizing high-assurance engineering, allowing architects to construct self-driving quality gates that automatically adapt to the system's emergent topology. Ultimately, this research suggests that the next frontier of cloud-native security lies not in the depth of individual scans, but in the horizontal integration of evidence. By treating assurance as a programmable, multi-source signal fusion problem, we provide the foundation for resilient microservice systems that are not only scalable by design but *continuously* verifiable by construction.

Acknowledgment

This material is based upon work supported by the National Science Foundation under Grant No. 2409933. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

References

[1] M. Loukides and S. Swoyer, "Microservices adoption in 2020," https://www.oreilly.com/radar/microservices-adoption-in-2020/, 2020.

[2] L. Lelovic, A. Huzinga, G. Goulis, A. Kaur, R. Boone, U. Muzrapov, A. S. Abdelfattah, and T. Cerny, "Change impact analysis in microservice systems: A systematic literature review," *Journal of Systems and Software*, vol. 219, p. 112241, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121224002851

[3] T. Cerny, A. S. Abdelfattah, A. A. Maruf, A. Janes, and D. Taibi, "Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study," *Journal of Systems and Software*, vol. 206, p. 111829. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0164121223002248

[4] R. Xin, P. Chen, and Z. Zhao, "CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications," *Journal of Systems and Software*, vol. 203, p. 111724, 2023. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S016412122300119X

[5] J. Bogner, J. Fritzsch, S. Wagner, and A. Zimmermann, "Industry practices and challenges for the evolvability assurance of microservices," *Empirical Software Engineering*, vol. 26, no. 5, p. 104, 2021. [Online]. Available: https://doi.org/10.1007/s10664-021-09999-9

[6] P. Chen, Y. Song, and Y. Xia, "Adaptively diagnosing system faults in microservice architecture: An autonomous predictive model construction framework," *Future Generation Computer Systems*, vol. 177, p. 108256, 2026. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0167739X25005503

[7] J. Kraft. (2025) Technical architecture analyses of major tech companies. [Online]. Available: https://jennykraft.de/deep-research/software-architectures/

[8] X. Zhou, S. Li, L. Cao, H. Zhang, Z. Jia, C. Zhong, Z. Shan, and M. A. Babar, "Revisiting the practices and pains of microservice architecture in reality: An industrial inquiry," *Journal of Systems and Software*, vol. 195, p. 111521, 2023. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S0164121222001972

[9] T. Cerny, G. Goulis, and A. S. Abdelfattah, "Towards change impact analysis in microservices-based system evolution," in *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2025, pp. 159–169. [Online]. Available: https://ieeexplore.ieee.org/document/10992369/

[10] A. Arcuri, "EvoMaster: Evolutionary multi-context automated system test generation," in *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2018, pp. 394–397. [Online]. Available: https://ieeexplore.ieee.org/document/8367066/

[11] D. Corradini, A. Zampieri, M. Pasqua, and M. Ceccato, "RestTestGen: An extensible framework for automated black-box testing of RESTful APIs," in *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2022, pp. 504–508. [Online]. Available: https://ieeexplore.ieee.org/document/9978261/

[12] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 748–758. [Online]. Available: https://ieeexplore.ieee.org/document/8811961/

[13] S. Weerasinghe and T. Cerny, "A neuro-symbolic risk calculus for quantifying security posture in microservice systems." ACM, June 2026.

[14] T. Cerny, G. Goulis, S. Perry, M. Edmonds, A. S. Abdelfattah, M. Esposito, A. Bakhtin, V. Lenarduzzi, and D. Taibi, "Analyzing evolution of microservice-based systems," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 2025, pp. 1030–1034. [Online]. Available: https://dl.acm.org/doi/10.1145/3696630.3728575

[15] C. Wojtak, "Formal methods for verifying authorization policy in microservice systems," in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 2025, pp. 241–242. [Online]. Available: https://ieeexplore.ieee.org/document/11126128/

[16] M. Schiewe, J. Curtis, V. Bushong, and T. Cerny, "Advancing static code analysis with language-agnostic component identification," *IEEE Access*, vol. 10, pp. 30 743–30 761, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9737496/

[17] A. Jøsang, *Subjective Logic*, ser. Artificial Intelligence: Foundations, Theory, and Algorithms. Cham, Switzerland: Springer International Publishing, 2016.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 15:15:25 UTC from IEEE Xplore. Restrictions apply.